Automata Theory Answers

1. Define Turing Machine with Formal Definition and Diagram

Definition

A Turing Machine (TM) is an abstract computational model consisting of a finite control, an infinite tape, and a read/write head. It can read symbols, write symbols, move left or right on the tape, and change states.

Formal Definition

A Turing Machine is represented by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where

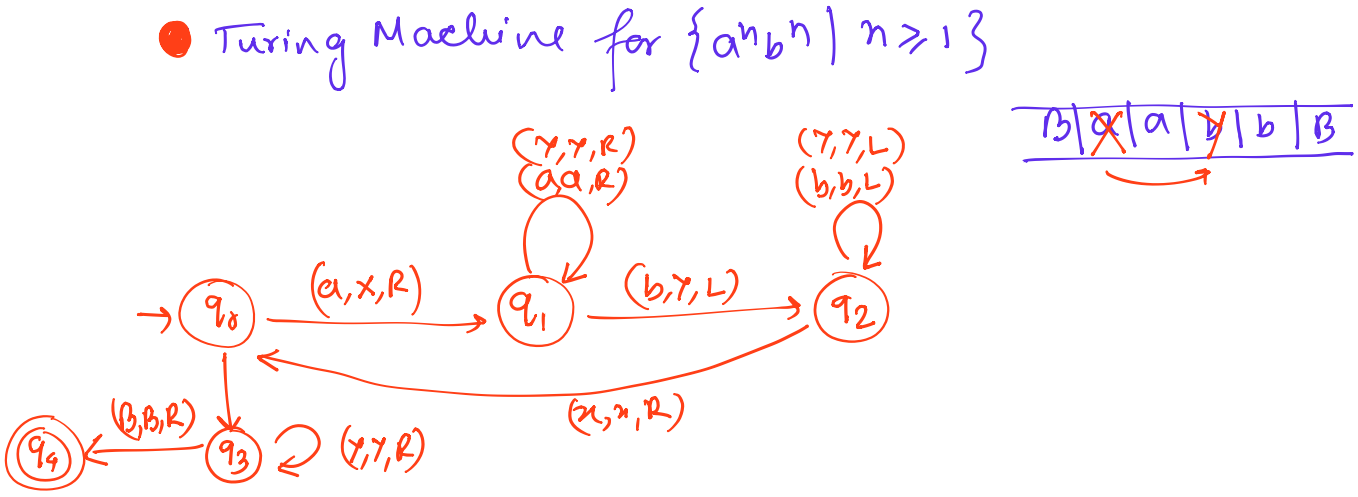
- Q = finite set of states
- Σ = input alphabet
- Γ = tape alphabet where $\Sigma \subseteq \Gamma$
- δ = transition function

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

- q_0 = initial state
- B = blank symbol
- F = set of final states

Conclusion

A Turing Machine can perform any computation that can be algorithmically described and is more powerful than DFA and PDA.



2. What is Universal Turing Machine?

Definition

A Universal Turing Machine (UTM) is a Turing Machine that can simulate the operation of any other Turing Machine.

Working

The UTM takes as input:

1. Description (encoding) of a TM M
2. Input string w

and simulates M on w.

Representation

$$UTM(\langle M \rangle, w)$$

where $\langle M \rangle$ is the encoded description of machine M.

Importance

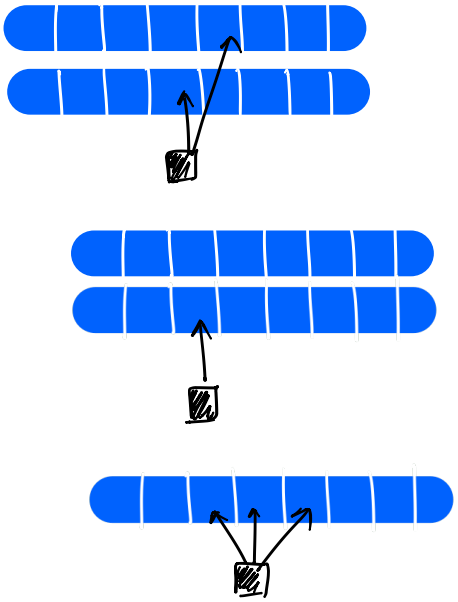
- Basis of modern computers.
- Can execute any program when given its description.
- Stores both program and data in memory.

Conclusion

A Universal TM acts like a general-purpose computer capable of simulating any Turing Machine.

3. Describe Variants of Turing Machines and Their Equivalence

Variants of Turing Machines



(i) Multi-Tape Turing Machine

- Has multiple tapes and heads.
- Faster computation.

(ii) Multi-Track Turing Machine

- Single tape divided into multiple tracks.
- Several symbols stored in one cell.

(iii) Multi-Head Turing Machine

- One tape with several heads.
- Heads move independently.

(iv) Non-Deterministic Turing Machine

- Multiple possible moves from a state.

(v) Two-Dimensional Turing Machine

- Head moves Left, Right, Up and Down.

(vi) Universal Turing Machine

- Simulates all other Turing Machines.

Equivalence

All these variants are equivalent in computational power to the standard Turing Machine.

$$L(MultiTape) = L(MultiHead) = L(NTM) = L(Standard\ TM)$$

They may differ in efficiency but recognize the same class of languages.

Conclusion

All variants accept exactly the recursively enumerable languages.

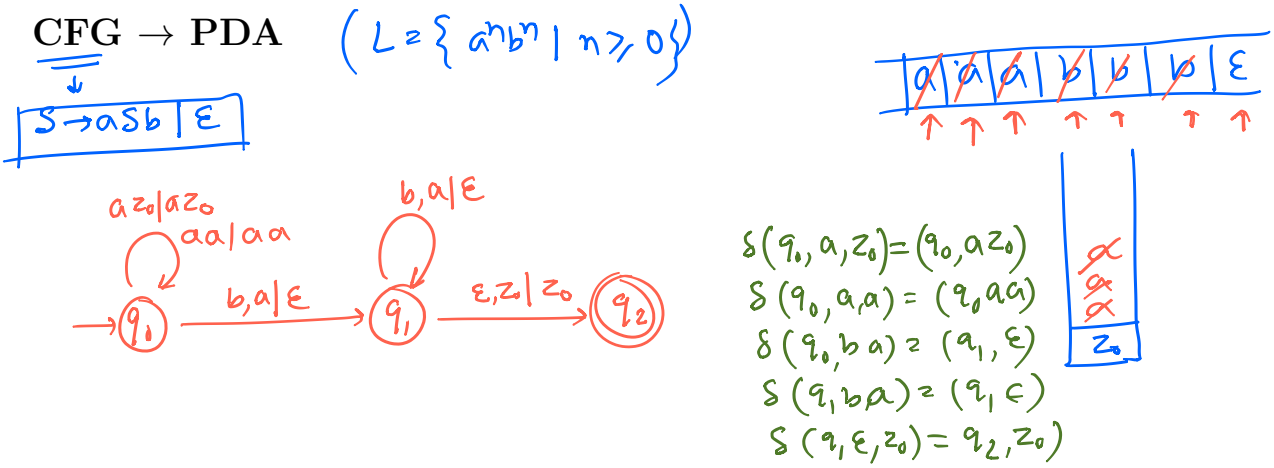
4. Show Equivalence Between CFG and PDA

Theorem

A language is Context-Free if and only if it is accepted by a Pushdown Automaton.

$$CFG \iff PDA$$

CFG → PDA



PDA → CFG

Conclusion

$$Language(CFG) = Language(PDA)$$

Therefore CFG and PDA are equivalent models for Context-Free Languages.

5. Explain DPDA vs NPDA

DPDA	**NPDA**
Deterministic PDA	Non-Deterministic PDA
Only one move possible	Multiple moves possible
No ambiguity in transitions	Several choices allowed
Less powerful	More powerful
Accepts DCFLs	Accepts all CFLs

Transition Function

For DPDA:

$$\delta(q, a, X)$$

gives at most one result.
For NPDA:

$$\delta(q, a, X)$$

can give multiple results.

Example

$$L = \{ww^R \mid w \in \{a, b\}^*\}$$

can be accepted by NPDA but not by DPDA.

Conclusion

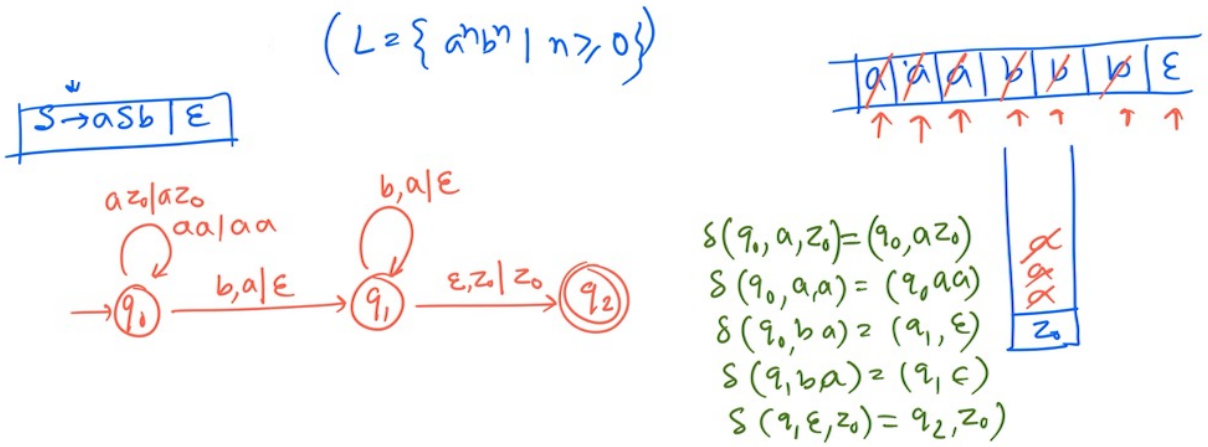
$$DPDA \subset NPDA$$

Therefore NPDA is strictly more powerful than DPDA.

6. Construct PDA for $L = \{a^n b^n \mid n \geq 0\}$

Idea

- Push one symbol A for every a.
- Pop one symbol A for every b.
- Accept when the stack returns to the bottom symbol.



7. Check Whether They Are Equivalent or Not

To check whether two automata or two regular expressions are equivalent:

1. Construct the equivalent DFA for both.
2. Minimize both DFAs.
3. Compare the minimized DFAs.

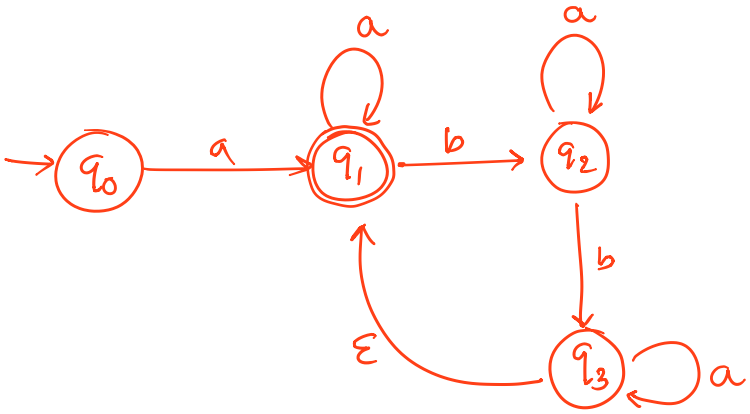
If both minimized DFAs are identical, then the automata are equivalent.
Conclusion:
Two automata are equivalent if they accept exactly the same language.

8. Design an NFA for the Regular Expression

$\{ a(a^* ba^* ba^*)^* \}$

$R = a(a^* ba^* ba^*)^*$

$L = \{ abaaba, aabbaa, aaabbbaaa \cdots \}$



NFA

$Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{a, b\}$, $^{start} = q_0$, $F = \{q_1\}$

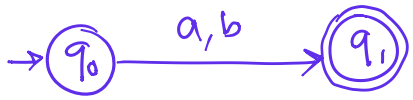
Transition function

$\delta(q_0, a) = \{q_1\}$
$\delta(q_0, b) = \phi$
$\delta(q_1, a) = \{q_1\}$
$\delta(q_1, b) = \{q_2\}$
$\delta(q_2, a) = \{q_2\}$
$\delta(q_2, b) = \{q_3\}$
$\delta(q_3, a) = \{q_3\}$
$\delta(q_3, \varepsilon) = \{q_1\}$

9(a). Finite Automaton for $a + b$

$L = \{a, b\}$

NFA

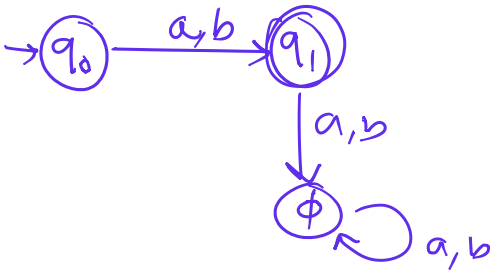


$Q = \{q_0, q_1\}$
$\Sigma = \{a, b\}$
$F = \{q_1\}$

Transitions:

$\delta(q_0, a) = \{q_1\}$
$\delta(q_0, b) = \{q_1\}$
all other transitions ϕ

DFA



$M = \{Q, \Sigma, \delta, q_0, F\}$
$Q = \{q_0, q_1, q_2\}$
$\Sigma = \{a, b\}$, $F = q_1$
start = q_0

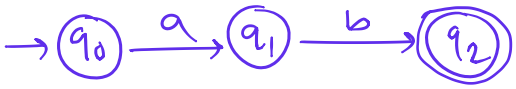
Transitions

$\delta(q_0, a) = \{q_1\}$
$\delta(q_0, b) = \{q_1\}$
$\delta(q_1, a) = \{\phi\}$
$\delta(q_1, b) = \{\phi\}$
$\delta(\phi, a) = \{\phi\}$
$\delta(\phi, b) = \{\phi\}$

δ	a	b
q_0	q_1	q_1
q_1	ϕ	ϕ
ϕ	ϕ	ϕ

9(b). Finite Automaton for ab

NFA



$M = \{Q, \Sigma, \delta, q_0, F\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$q_0 \rightarrow$ Start

$F = \{q_2\}$

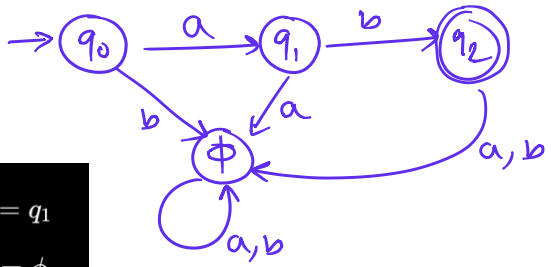
Transition fn.

$$\delta(q_0, a) = \{q_1\}$$

$$\delta(q_1, b) = \{q_2\}$$

δ	a	b
q_0	$\{q_1\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

DFA



$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, \phi\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \text{Start State}$$

$$F = \{q_2\}$$

$$\delta(q_0, a) = q_1$$
$$\delta(q_0, b) = \phi$$
$$\delta(q_1, a) = \phi$$
$$\delta(q_1, b) = q_2$$
$$\delta(q_2, a) = \phi$$
$$\delta(q_2, b) = \phi$$
$$\delta(\phi, a) = \phi$$
$$\delta(\phi, b) = \phi$$

State	a	b
q_0	q_1	ϕ
q_1	ϕ	q_2
q_2	ϕ	ϕ
ϕ	ϕ	ϕ

9(c). Finite Automaton for

$$ab + (b + aa)b^*a$$

NFA

$R = ab + (b + aa)\,b^*a$

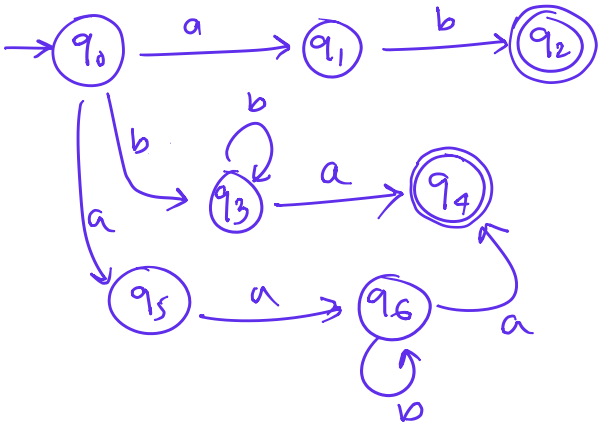
ab or. $(b$ or $aa)$ b. must- a

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \text{Start State}$$

$$F = \{q_2, q_4\}$$



$$\delta(q_0, a) = \{q_1, q_5\}$$

$$\delta(q_0, b) = \{q_3\}$$

$$\delta(q_1, b) = \{q_2\}$$

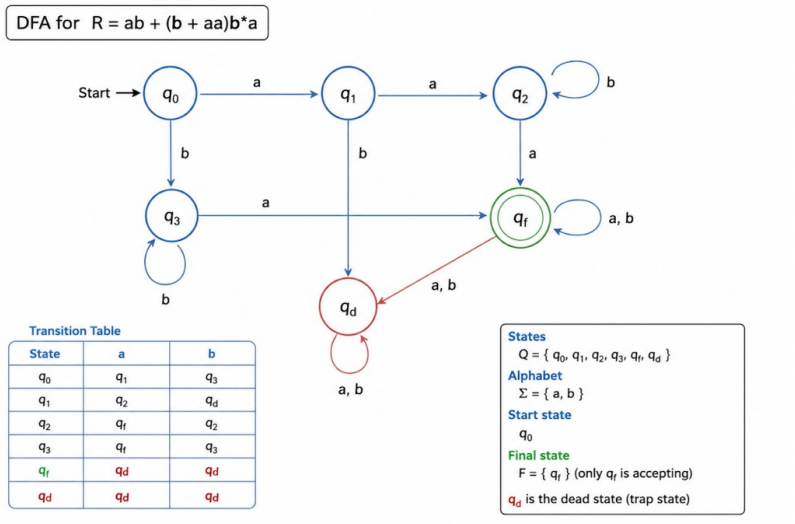
$$\delta(q_3, b) = \{q_3\}$$

$$\delta(q_3, a) = \{q_4\}$$

$$\delta(q_5, a) = \{q_6\}$$

$$\delta(q_6, b) = \{q_6\}$$

$$\delta(q_6, a) = \{q_4\}$$



10. What are Regular Expressions?

A Regular Expression (RE) is a formal notation used to describe regular languages.
Regular expressions are formed using:

- Union (+)
- Concatenation
- Kleene Star (*)

Examples

$$a + b$$

represents

$$\{a, b\}$$

$$ab$$

represents

$$\{ab\}$$

$$a^*$$

represents

$$\{\epsilon, a, aa, aaa, \ldots\}$$

Applications

- Lexical analysis
- Pattern matching
- Text searching
- Compiler design

Conclusion:
Regular expressions are algebraic representations of regular languages.

11. Prove That

$$0^*1(0+10^*1)^*$$

is equal to

$$(1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)$$

Proof

Let

$$R = 0^*1(0+10^*1)^*$$

Observe that

$$0^*1 = 1+00^*1$$

Therefore,

$$R = (1+00^*1)(0+10^*1)^*$$

Using the identity

$$A^* = \epsilon + A^*A$$

we get

$$R = (1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)$$

which is exactly

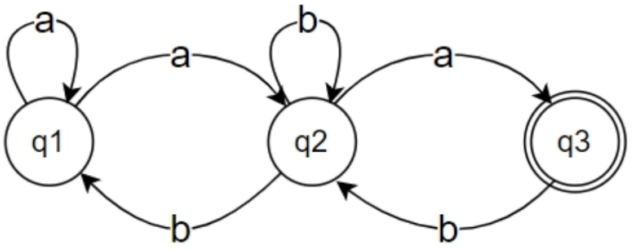
$$(1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)$$

Hence proved.

$$0^*1(0+10^*1)^* = (1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)$$

Q.E.D.

12. Convert the following NFA to a regular expression.



$q_1 = q_1 a + q_2 b + \varepsilon$

$q_2 = q_1 a + q_2 b + q_3 b$

$q_3 = q_2 a$

$R = Q + RP \rightarrow R = QP^*$

$q_2 = q_1 a + q_2 b + q_2 ab = q_1 a + q_2 (ab + b) = q_1 a (ab + b)^*$

$q_1 = q_1 a + q_2 b + \varepsilon = q_1 a + q_1 a (ab+b)^* b + \varepsilon$

$= q_1 (a + a (ab+b)^* b) + \varepsilon$

$= \varepsilon + q_1 (a + a (ab+b)^* b)$

$q_1 = (a + a (ab+b)^* b)^*$

$\therefore q_2 = q_1 a (ab+b)^* = (a + a (ab+b)^* b)^* a (ab+b)^*$

$q_3 = q_2 a = (a + a (ab+b)^* b)^* a (ab+b)^* a$

13. What is an Equivalent DFA?

Definition

An Equivalent DFA (Deterministic Finite Automaton) is a DFA that accepts exactly the same language as a given automaton (NFA, ϵ-NFA, or another DFA).

If two automata accept the same set of strings, they are said to be equivalent.

$$L(DFA_1) = L(DFA_2)$$

where $L(DFA)$ denotes the language accepted by the DFA.

Example

Consider the language

$$L = \{w \in \{0,1\}^* \mid w \text{ ends with } 1\}$$

An NFA and a DFA may have different numbers of states, but if both accept all strings ending in 1, then they are equivalent.

How to Check Equivalence

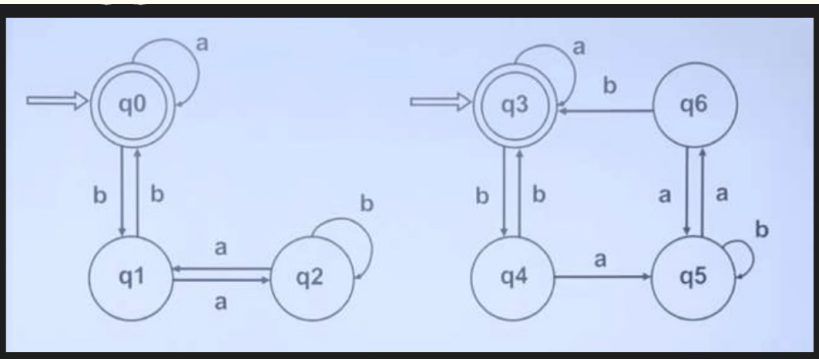
1. Convert both automata into DFAs.
2. Minimize the DFAs.
3. Compare the minimized DFAs.

If the minimized DFAs are identical, then the automata are equivalent.

Conclusion

Two DFAs are equivalent if and only if they accept exactly the same language, even if their structures or numbers of states are different.

14. Check whether they are equivalent or not.



DFA 1 DFA 2

Steps

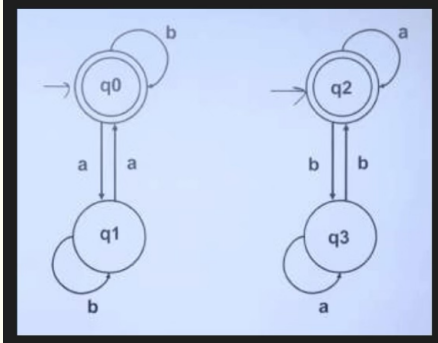
① Same Initial & Final State ✓

② (q_0, q_3) transition

	a	b
(q_0, q_3)	$q_0 q_3$ FS FS	$q_1 q_4$ IS IS
(q_1, q_4)	$q_2 q_5$ IS IS	$q_0 q_3$ FS FS
(q_2, q_5)	$q_1 q_6$ IS IS	$q_2 q_5$ IS IS
(q_1, q_6)	$q_2 q_5$ IS IS	$q_0 q_3$ FS FS

∴ We can say that all states matched.

DFA 1 ≡ DFA 2



DFA 1 DFA 2

① Initial & final state same ✓

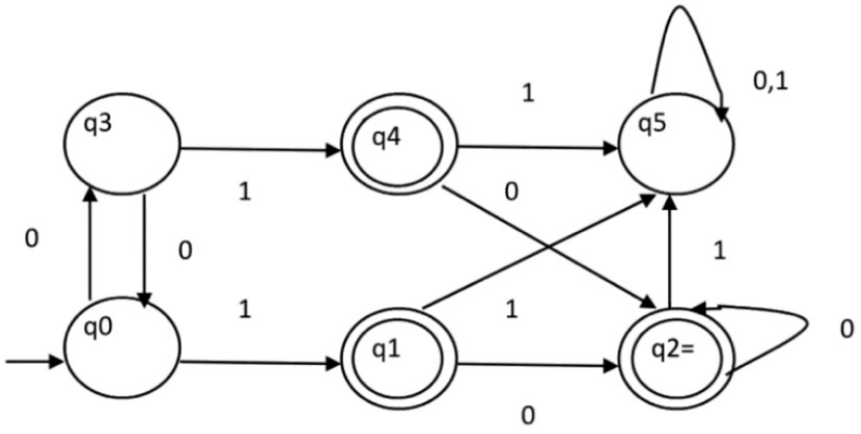
② (q_0, q_2)

	a	b
(q_0, q_2)	$q_1 q_2$ IS FS ✗	$q_0 q_3$ FS IS ✗

∴ DFA 1 ≠ DFA 2

Conflict

15. Minimize the following DFA



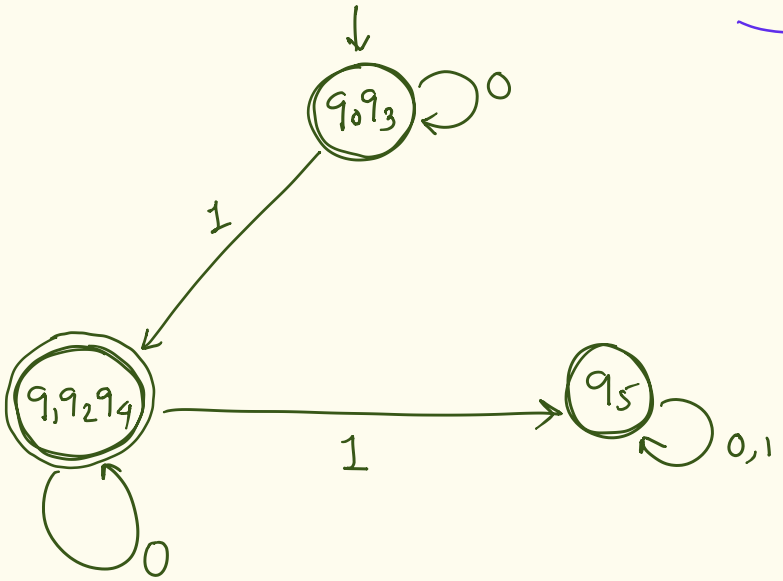
	0	1
q_0	q_3	q_1
q_1	q_2	q_5
q_2	q_2	q_5
q_3	q_0	q_4
q_4	q_2	q_5
q_5	q_5	q_5

① Unreachable states → Nothing

② Convert states to classes.

	Final	Non Final	
π_0	$\{q_1, q_2, q_4\}$ G_1	$\{q_0, q_3, q_5\}$ G_2	
π_1	$\{q_1 q_2 q_4\}$ G_1	$\{q_0, q_3\}$ G_{21}	$\{q_5\}$ G_{22}
π_3	$\{q_1 q_2 q_4\}$	$\{q_0 q_3\}$	$\{q_5\}$

total 3 states.



16. Explain Parse Trees and Their Relation to Derivations

Parse Tree

A Parse Tree (Derivation Tree) is a tree representation of the derivation of a string according to a Context-Free Grammar (CFG).

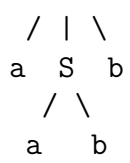
- Root node is the start symbol.
- Internal nodes are non-terminals.
- Leaf nodes are terminals or ϵ.

Example

Grammar:

$$S \to aSb \mid ab$$

String: *aabb*



Relation to Derivations

A parse tree represents both leftmost and rightmost derivations.

Leftmost Derivation:

$$S \Rightarrow aSb \Rightarrow a(ab)b \Rightarrow aabb$$

Rightmost Derivation:

$$S \Rightarrow aSb \Rightarrow aabb \Rightarrow aabb$$

Conclusion

Every parse tree corresponds to at least one leftmost and one rightmost derivation.

17. Regular Grammar vs Context-Free Grammar

Regular Grammar	**Context-Free Grammar**
Type-3 Grammar	Type-2 Grammar
Recognized by FA	Recognized by PDA
Less powerful	More powerful
Production: $A \rightarrow aB$ or $A \rightarrow a$	Production: $A \rightarrow \alpha$
Generates Regular Languages	Generates CFLs

Example

Regular Grammar:

$$S \rightarrow aS \mid b$$

CFG:

$$S \rightarrow aSb \mid \epsilon$$

Conclusion

Every Regular Grammar is a CFG but every CFG is not a Regular Grammar.

18. Discuss Ambiguity in Grammars with Examples

Definition

A grammar is ambiguous if a string generated by it has more than one parse tree or more than one leftmost/rightmost derivation.

Example

Grammar:

$$E \to E + E \mid E * E \mid id$$

String:

$$id + id * id$$

This string can be parsed as:

$$(id + id) * id$$

or

$$id + (id * id)$$

Hence two different parse trees exist.

Conclusion

Therefore the grammar is ambiguous.

19. Generate Strings for

$$L = \{\text{all strings over } \{0, 1\} \text{ containing } 00\}$$

Examples:

00

001

100

000

1100

0011

1001

11001

All strings contain the substring 00.

Conclusion

The language consists of all binary strings having at least one occurrence of 00.

20. Construct an NFA and Convert it to Equivalent DFA

$R = ab$

NFA

$M = \{Q, \Sigma, \delta, q_0, F\}$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$q_0 \rightarrow$ Start

$F = \{q_2\}$

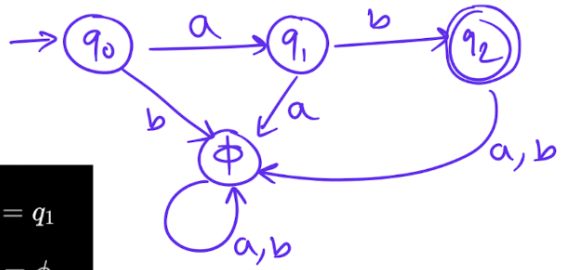
Transition fn.

$$\delta(q_0, a) = \{q_1\}$$

$$\delta(q_1, b) = \{q_2\}$$

δ	a	b
q_0	$\{q_1\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

DFA



$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, \phi\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \text{Start State}$$

$$F = \{q_2\}$$

$$\delta(q_0, a) = q_1$$

$$\delta(q_0, b) = \phi$$

$$\delta(q_1, a) = \phi$$

$$\delta(q_1, b) = q_2$$

$$\delta(q_2, a) = \phi$$

$$\delta(q_2, b) = \phi$$

$$\delta(\phi, a) = \phi$$

$$\delta(\phi, b) = \phi$$

State	a	b
q_0	q_1	ϕ
q_1	ϕ	q_2
q_2	ϕ	ϕ
ϕ	ϕ	ϕ

21. Explain Chomsky Hierarchy

Type	Grammar	Machine
0	Unrestricted	Turing Machine
1	Context Sensitive	LBA
2	Context Free	PDA
3	Regular	DFA/NFA

Examples:
Type-3:

$$S \to aS \mid b$$

Type-2:

$$S \to aSb \mid \epsilon$$

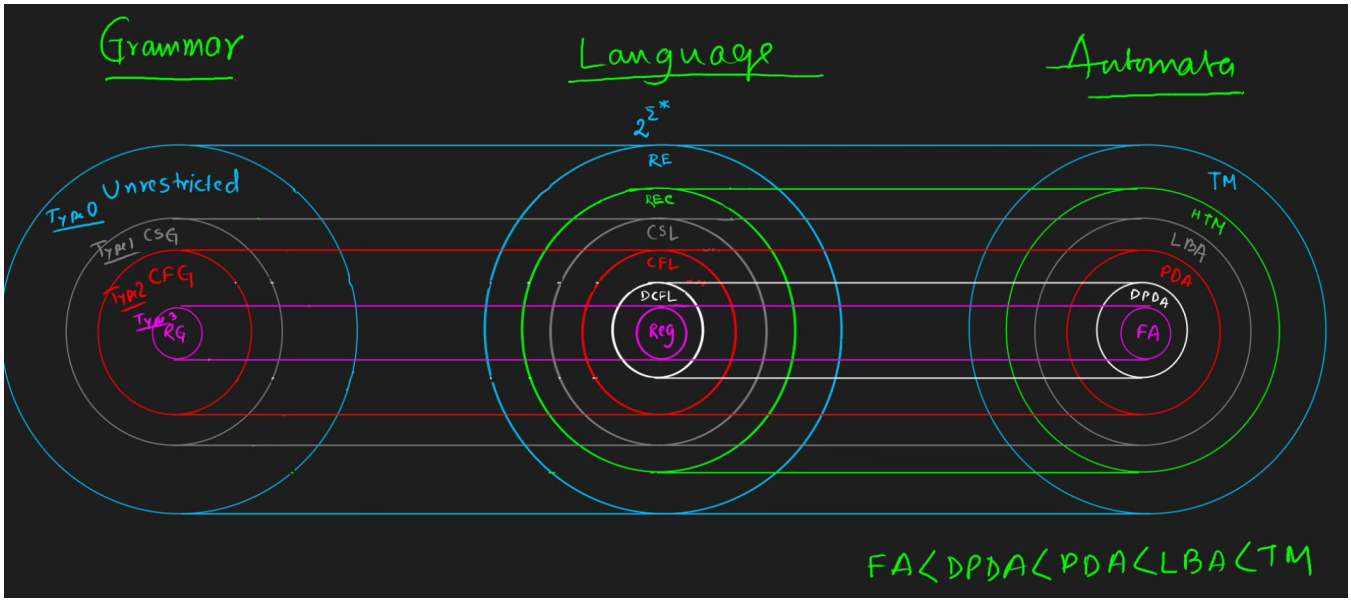
Type-1:

$$aSb \to ab$$

Type-0:
No restrictions.

Hierarchy

$$Regular \subset CFL \subset CSL \subset RE$$



22. Operations on Languages

Let

$$L_1 = \{a, b\}$$

$$L_2 = \{b, c\}$$

Union

$$L_1 \cup L_2 = \{a, b, c\}$$

Intersection

$$L_1 \cap L_2 = \{b\}$$

Difference

$$L_1 - L_2 = \{a\}$$

Concatenation

$$L_1 L_2 = \{ab, ac, bb, bc\}$$

Kleene Star

$$L^* = \{\epsilon, a, b, aa, ab, \ldots\}$$

23. Differentiate Between DFA and Turing Machine

DFA	**Turing Machine**
Finite memory	Infinite tape
Read only input	Read/write tape
Cannot modify input	Can modify tape contents
Accepts Regular Languages	Accepts RE Languages
Less powerful	More powerful

Conclusion

A Turing Machine is computationally more powerful than a DFA.

24. Define a Turing Machine and Explain its Components

A Turing Machine is represented by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

Components:

- Q : Set of states
- Σ : Input alphabet
- Γ : Tape alphabet
- δ : Transition function
- q_0 : Start state
- B : Blank symbol
- F : Final states

It consists of an infinite tape and a read/write head.

25. Construct PDA for

$$L = \{a^n b^n \mid n \geq 1\}$$

Idea:

- Push one A for every a.
- Pop one A for every b.
- Accept when stack becomes empty.

States:

$$Q = \{q_0, q_1, q_f\}$$

Stack Symbols:

$$\Gamma = \{Z_0, A\}$$

Transitions:

$$(q_0, a, Z_0) \to (q_0, AZ_0)$$

$$(q_0, a, A) \to (q_0, AA)$$

$$(q_0, b, A) \to (q_1, \epsilon)$$

$$(q_1, b, A) \to (q_1, \epsilon)$$

$$(q_1, \epsilon, Z_0) \to (q_f, Z_0)$$

26. Define Pushdown Automaton (PDA) and Explain its Components

Definition

A Pushdown Automaton (PDA) is a finite automaton equipped with a stack.

Formal Definition

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where

- Q : Finite set of states
- Σ : Input alphabet
- Γ : Stack alphabet
- δ : Transition function
- q_0 : Initial state
- Z_0 : Initial stack symbol
- F : Final states

Components

1. Input tape
2. Finite control
3. Stack

Conclusion

A PDA recognizes Context-Free Languages and is more powerful than a finite automaton.

27. Explain Derivation in CFG. Differentiate Between Leftmost and Rightmost Derivation

Derivation in CFG

A derivation is a sequence of production-rule applications used to generate a string from the start symbol.

If

$$S \Rightarrow \alpha$$

then α is derived from S.

Example

Grammar:

$$S \to aSb \mid ab$$

Generate string $aabb$:

$$S \Rightarrow aSb \Rightarrow a(ab)b \Rightarrow aabb$$

Leftmost Derivation

At each step, the leftmost non-terminal is replaced.

$$S \Rightarrow aSb \Rightarrow a(ab)b \Rightarrow aabb$$

Rightmost Derivation

At each step, the rightmost non-terminal is replaced.

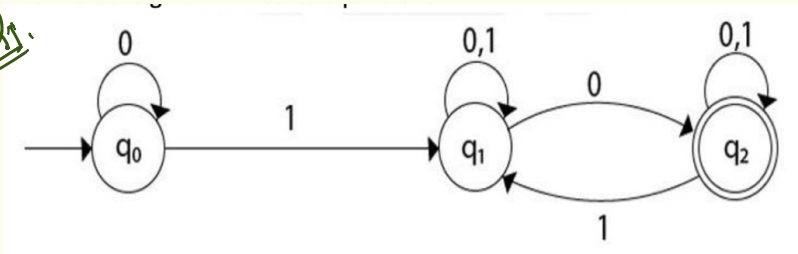
$$S \Rightarrow aSb \Rightarrow aabb \Rightarrow aabb$$

Difference

Leftmost Derivation	Rightmost Derivation
Expand leftmost non-terminal first	Expand rightmost non-terminal first
Denoted by $\Rightarrow_{lm}$	Denoted by $\Rightarrow_{rm}$
Used in top-down parsing	Used in bottom-up parsing

28. NFA → DFA

Q1.



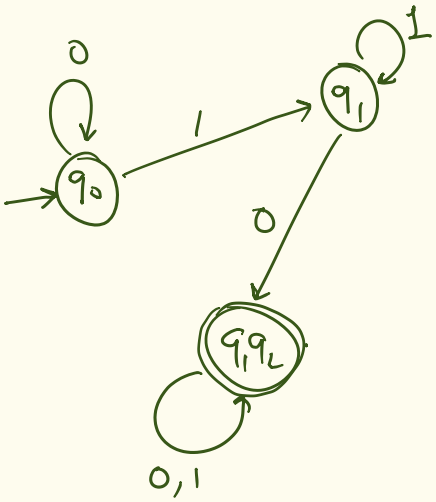
Step 1: NFA Transition Table

From the diagram:

State	0	1
q_0	$\{q_0\}$	$\{q_1\}$
q_1	$\{q_1, q_2\}$	$\{q_1\}$
q_2	$\{q_2\}$	$\{q_1, q_2\}$

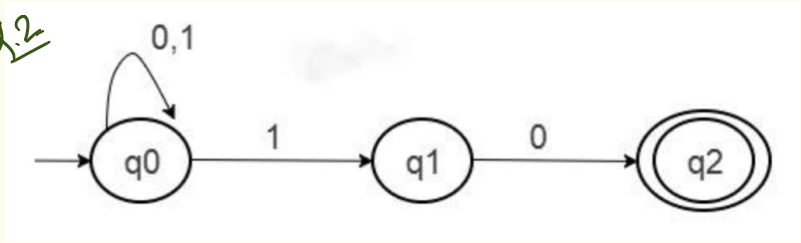
DFA transition table

	0	1
q_0	q_0	q_1
q_1	q_1q_2	q_1
q_1q_2	q_1q_2	q_1q_2



Final state = $\{q_1q_2\}$

Q.2

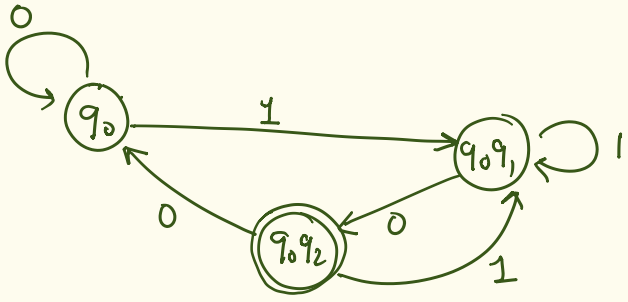


NFA Table

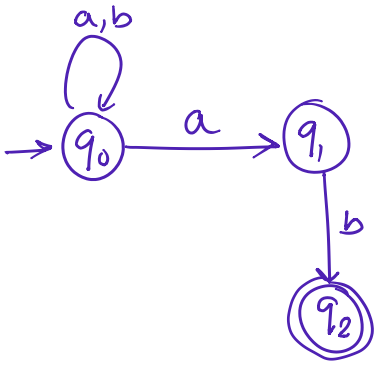
	0	1
q_0	q_0	q_0q_1
q_1	q_2	ε
q_2	ε	ε

DFA table

	0	1
q_0	q_0	q_0q_1
q_0q_1	q_0q_2	q_0q_1
q_0q_2	q_0	q_0q_1



29. Convert the Regular Expression $(a+b)^*ab$ into an Equivalent DFA



The expression

$$(a+b)^*ab$$

represents all strings over $\{a,b\}$ ending with ab.

States:

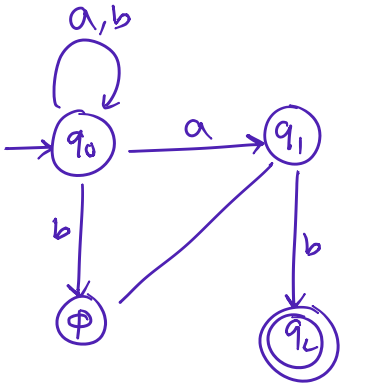
$$Q = \{q_0, q_1, q_2\}$$

Start State:

$$q_0$$

Final State:

$$q_2$$



Transition Table:

State	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

Closure Under Intersection

Let L_1 and L_2 be regular languages.

Suppose:

$$M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$$

$$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$$

Construct product automaton:

$$M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$$

where

$$\delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a))$$

Then

$$L(M) = L_1 \cap L_2$$

Hence regular languages are closed under intersection.

30. DFA Accepting Strings Containing Neither 00 Nor 11

Language:

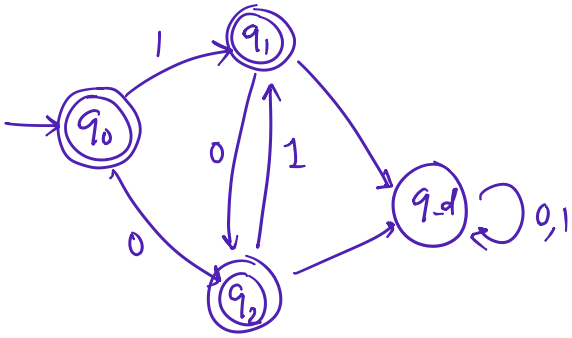
$$L = \{w \in \{0,1\}^* \mid w \text{ contains neither 00 nor 11}\}$$

The symbols must alternate.
Accepted strings:

$$\epsilon, 0, 1, 01, 10, 010, 101, \ldots$$

States:

$$Q = \{q_0, q_1, q_2, q_d\}$$



where q_d is dead state.
Transition Table:

State	0	1
q_0	q_1	q_2
q_1	q_d	q_2
q_2	q_1	q_d
q_d	q_d	q_d

Final States:

$$\{q_0, q_1, q_2\}$$

31. Which Automaton Has Ambiguity?

Answer

NFA is ambiguous because multiple transitions for the same input symbol may exist.
Example:

$$\delta(q, a) = \{q_1, q_2\}$$

DFA is never ambiguous because exactly one transition exists for every input symbol.

$$\delta(q, a) = q_1$$

Conclusion

NFA may be ambiguous.
DFA is always unambiguous.

33. Define Finite Automaton and Differentiate Between DFA and NFA

Definition

A Finite Automaton (FA) is a mathematical model used to recognize regular languages. It is represented as

$$M = (Q, \Sigma, \delta, q_0, F)$$

Representations

1. Transition Diagram
2. Transition Table
3. 5-Tuple Representation

DFA vs NFA

DFA	NFA
One transition per symbol	Multiple transitions allowed
No ϵ transitions	ϵ transitions allowed
Deterministic	Non-deterministic

34. Definitions

(i) String

A finite sequence of symbols from an alphabet.
Example:

$$abba$$

(ii) Alphabet

A finite non-empty set of symbols.
Example:

$$\Sigma = \{a, b\}$$

(iii) Language

A set of strings over an alphabet.
Example:

$$L = \{a, ab, abb\}$$

(iv) Closure

A language class is closed under an operation if applying the operation still produces a language of the same class.

(v) Powerset

Set of all subsets of a set.
If

$$A = \{a, b\}$$

then

$$P(A) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$$

35. Construct Leftmost and Rightmost Derivations and Draw Parse Tree

Grammar:

$$S \to aSb \mid ab$$

String:

$$aabb$$

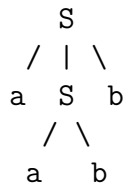
Leftmost:

$$S \Rightarrow aSb \Rightarrow a(ab)b \Rightarrow aabb$$

Rightmost:

$$S \Rightarrow aSb \Rightarrow aabb \Rightarrow aabb$$

Parse Tree:



36. Explain Types of Grammars in Chomsky Hierarchy

Type	Grammar
0	Unrestricted
1	Context Sensitive
2	Context Free
3	Regular

Hierarchy:

$$Regular \subset CFL \subset CSL \subset RE$$

Proof that Kleene Star Contains ϵ

Definition:

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Since

$$L^0 = \{\epsilon\}$$

therefore

$$\epsilon \in L^*$$

Hence Kleene Star always contains the empty string.